AI Assignment Submission

Problem Statement 0-B: Travelling Agent

Course	Artificial and Computational Intelligence
Assignment	Assignment 1
Problem	0-B
Student Name	Gadamsetty V Sai Manoj Kumar
BITS ID	2025MT12100

1. Agent Environment and PEAS

The agent moves from Chennai (A) to Hyderabad (X) over a weighted graph and reports route, cost, and complexity.

- Performance: minimize travel cost while reaching X.
- Environment: static, fully observable weighted graph.
- Actuators: move to adjacent nodes.
- Sensors: current node, neighbors, edge costs.

2. Heuristic / Fitness Function

h(n) is the average cost across all simple paths from n to X. IDA* uses f(n)=g(n)+h(n), Hill Climbing picks lowest-h neighbor.

3. Runtime Output (Markdown-Style Code Block)

````
```text
Assignment 1 - Problem Statement 0-B
Graph search from Chennai (A) to Hyderabad (X)

Heuristic values (average cost of all simple paths to X):
  h(A) = 22
  h(B) = 27
  h(C) = 31.3333
  h(D) = 19
  h(E) = 24.5
  h(F) = 21.6667
  h(G) = 20.3333
  h(H) = 31.6667
  h(X) = 0

IDA* result:
  Path: A -> D -> G -> X
  Total cost: 19
  Threshold progression: 22, 27.3333
  Nodes expanded: 5
  Max depth reached: 3
  Time complexity: O(b^d) in the worst case
  Space complexity: O(bd) in the worst case

Hill climbing result:
  Path: A -> D -> G -> X
  Total cost: 19
```
````

```
  Nodes expanded: 3
  Max depth reached: 3
  Reached goal: Yes
  Local optimum encountered: Yes
  Time complexity: O(b) per move
  Space complexity: O(b) for neighbor inspection
```

4. Python Source Code (Markdown-Style Code Block)

```python
from dataclasses import dataclass

from math import inf

from statistics import fmean

from typing import Dict, Iterable, List, Optional, Set, Tuple


Graph = Dict[str, Dict[str, int]]


GRAPH: Graph = {

    "A": {"B": 5, "D": 3},

    "B": {"A": 5},

    "C": {"G": 11},

    "D": {"A": 3, "E": 12, "F": 6, "G": 4},

    "E": {"D": 12, "F": 7},

    "F": {"D": 6, "E": 7, "H": 10, "X": 8},

    "G": {"C": 11, "D": 4, "X": 12},

    "H": {"F": 10},

    "X": {"F": 8, "G": 12},

}


START_NODE = "A"

GOAL_NODE = "X"


@dataclass

class SearchResult:

    path: List[str]

    cost: float

    nodes_expanded: int

    max_depth: int

    thresholds: Optional[List[float]] = None

    local_optimum: bool = False


def _all_simple_paths_with_costs(graph: Graph, start: str, goal: str) -> List[Tuple[List[str], int]]:

    paths: List[Tuple[List[str], int]] = []
```

```
    def dfs(node: str, current_path: List[str], current_cost: int, visited: Set[str]) -> None:
        if node == goal:
            paths.append((current_path.copy(), current_cost))
            return

        for neighbor, weight in graph[node].items():
            if neighbor in visited:
                continue
            visited.add(neighbor)
            current_path.append(neighbor)
            dfs(neighbor, current_path, current_cost + weight, visited)
            current_path.pop()
            visited.remove(neighbor)

    dfs(start, [start], 0, {start})
    return paths

def build_average_heuristic(graph: Graph, goal: str) -> Dict[str, float]:
    heuristic: Dict[str, float] = {}
    for node in graph:
        paths = _all_simple_paths_with_costs(graph, node, goal)
        heuristic[node] = fmean(cost for _, cost in paths) if paths else inf
    return heuristic

def hill_climbing(graph: Graph, start: str, goal: str, heuristic: Dict[str, float]) -> SearchResult:
    current = start
    path = [current]
    cost = 0.0
    visited = {current}
    nodes_expanded = 0
    max_depth = 0
    encountered_local_optimum = False

    while current != goal:
        nodes_expanded += 1
```

```
            neighbors = []
            for neighbor, edge_cost in graph[current].items():
                if neighbor in visited and neighbor != goal:
                    continue
                neighbors.append((heuristic[neighbor], edge_cost, neighbor))

            if not neighbors:
                return SearchResult(path=path, cost=cost, nodes_expanded=nodes_expanded, max_depth=max_dep

            neighbors.sort(key=lambda item: (item[0], item[1], item[2]))
            improving_neighbors = [item for item in neighbors if item[0] < heuristic[current]]

            if improving_neighbors:
                _, edge_cost, best_neighbor = improving_neighbors[0]
            else:
                encountered_local_optimum = True
                _, edge_cost, best_neighbor = neighbors[0]

            if best_neighbor in visited and best_neighbor != goal:
                return SearchResult(path=path, cost=cost, nodes_expanded=nodes_expanded, max_depth=max_dep

            current = best_neighbor
            path.append(current)
            visited.add(current)
            cost += edge_cost
            max_depth = max(max_depth, len(path) - 1)

        return SearchResult(path=path, cost=cost, nodes_expanded=nodes_expanded, max_depth=max_depth, loca

    def ida_star(graph: Graph, start: str, goal: str, heuristic: Dict[str, float]) -> SearchResult:
        thresholds: List[float] = []
        nodes_expanded = 0
        max_depth = 0

        def search(node: str, g_cost: float, threshold: float, path: List[str], visited: Set[str], depth:
```

```
        nonlocal nodes_expanded, max_depth
        max_depth = max(max_depth, depth)

        f_cost = g_cost + heuristic[node]
        if f_cost > threshold:
            return None, f_cost

        if node == goal:
            return path.copy(), g_cost

        nodes_expanded += 1
        minimum_excess = inf

        ordered_neighbors = sorted(
            ((g_cost + edge_cost + heuristic[neighbor], edge_cost, neighbor)
             for neighbor, edge_cost in graph[node].items() if neighbor not in visited),
            key=lambda item: (item[0], item[1], item[2])
        )

        for _, edge_cost, neighbor in ordered_neighbors:
            visited.add(neighbor)
            path.append(neighbor)
            found_path, next_bound = search(neighbor, g_cost + edge_cost, threshold, path, visited, de
            path.pop()
            visited.remove(neighbor)

            if found_path is not None:
                return found_path, _path_cost(graph, found_path)
            minimum_excess = min(minimum_excess, next_bound)

        return None, minimum_excess

    threshold = heuristic[start]
    while True:
        thresholds.append(threshold)
```

```
        found_path, result = search(start, 0.0, threshold, [start], {start}, 0)
        if found_path is not None:
            return SearchResult(path=found_path, cost=result, nodes_expanded=nodes_expanded, max_depth
        if result == inf:
            return SearchResult(path=[], cost=inf, nodes_expanded=nodes_expanded, max_depth=max_depth,
        threshold = result

def _path_cost(graph: Graph, path: List[str]) -> float:
    return sum(graph[left][right] for left, right in zip(path, path[1:]))

def _format_path(path: Iterable[str]) -> str:
    return " -> ".join(path)

def print_heuristic_table(heuristic: Dict[str, float]) -> None:
    print("Heuristic values (average cost of all simple paths to X):")
    for node in sorted(heuristic):
        value = heuristic[node]
        display = "inf" if value == inf else f"{value:.4f}".rstrip("0").rstrip(".")
        print(f"  h({node}) = {display}")
    print()

heuristic = build_average_heuristic(GRAPH, GOAL_NODE)
ida_result = ida_star(GRAPH, START_NODE, GOAL_NODE, heuristic)
hill_result = hill_climbing(GRAPH, START_NODE, GOAL_NODE, heuristic)

print("Assignment 1 - Problem Statement 0-B")
print("Graph search from Chennai (A) to Hyderabad (X)")
print()
print_heuristic_table(heuristic)

print("IDA* result:")
print(f"  Path: {_format_path(ida_result.path)}")
print(f"  Total cost: {ida_result.cost:.0f}")
print(f"  Threshold progression: {', '.join(f'{value:.4f}'.rstrip('0').rstrip('.') for value in ida_re
print(f"  Nodes expanded: {ida_result.nodes_expanded}")
```

```
print(f"  Max depth reached: {ida_result.max_depth}")
print("  Time complexity: O(b^d) in the worst case")
print("  Space complexity: O(bd) in the worst case")
print()

print("Hill climbing result:")
print(f"  Path: {_format_path(hill_result.path)}")
print(f"  Total cost: {hill_result.cost:.0f}")
print(f"  Nodes expanded: {hill_result.nodes_expanded}")
print(f"  Max depth reached: {hill_result.max_depth}")
print(f"  Reached goal: {'Yes' if hill_result.path and hill_result.path[-1] == GOAL_NODE else 'No'}")
print(f"  Local optimum encountered: {'Yes' if hill_result.local_optimum else 'No'}")
print("  Time complexity: O(b) per move")
print("  Space complexity: O(b) for neighbor inspection")
```